



SENIOR .NET INTERVIEW SERIES

ASP.NET Core

Middleware, DI lifetimes, minimal APIs, validation — the framework questions you must nail cold.

20 Real Interview Questions, Answered

Swipe through — save this for your next interview prep.

Q. Explain the middleware pipeline. Why does order matter?

A. Each request flows through a chain of middleware; each can act, call the next, then act again on the way back — an onion. Order is the contract: exception handling first to catch everything below, then HSTS/HTTPS, static files (short-circuit early), routing, CORS, authentication before authorization, then endpoints. Auth after endpoints simply never runs.

Q. Transient vs Scoped vs Singleton — and the classic trap?

A. Transient: new instance per resolve. Scoped: one per request. Singleton: one for the app lifetime. The trap is captive dependencies: a singleton injecting a scoped service captures it forever — with DbContext that means a shared, non-thread-safe context. Fix: inject IServiceScopeFactory and create scopes explicitly in singletons and background services.

Q. Minimal APIs vs controllers — how do you choose in 2026?

A. Minimal APIs: less ceremony, better performance, first-class AOT support, and filters/validation have matured — great for microservices and focused endpoints, organized via route groups. Controllers still win for large teams wanting convention, model binding edge cases, and existing filter pipelines. It's a team/consistency decision more than a technical one now.

Q. How does model binding and validation actually flow?

A. Binding pulls values from route, query, headers, and body (JSON deserialization) into parameters or models. Validation then runs — data annotations or FluentValidation — populating ModelState. [ApiController] auto-returns 400 with ProblemDetails on invalid state. Senior points: validate at the edge, keep domain invariants in the domain, and .NET 10 brings validation to minimal APIs natively.

Q. IOptions vs IOptionsSnapshot vs IOptionsMonitor?

A. `IOptions<T>`: singleton, read once, never updates. `IOptionsSnapshot<T>`: scoped, re-evaluated per request — picks up config changes between requests. `IOptionsMonitor<T>`: singleton with change notifications via `OnChange` — the only choice inside singletons and background services that must react to config reloads. Bind once, validate with `ValidateOnStart`.

Q. What's the difference between `app.Use`, `app.Run`, and `app.Map`?

A. `Use` adds middleware that can call the next delegate. `Run` is terminal — it never calls next, ending the pipeline. `Map` branches the pipeline by path prefix, so requests to `/admin` can flow through a different chain. Interviewers use this to check you understand the pipeline is just composed delegates.

```
app.Use(async (ctx, next) => {  
    // before  
    await next(ctx);  
    // after (response path)  
});
```

Q. How do exception handling and ProblemDetails work together?

A. UseExceptionHandler catches unhandled exceptions; since .NET 8 you implement IExceptionHandler for typed handling and return ProblemDetails (RFC 7807/9457) — a machine-readable error contract with type, title, status, and traceId. Never leak stack traces; log the full exception, return the sanitized ProblemDetails. One consistent error shape across the whole API.

Q. How does dependency injection construct your controller, end to end?

A. At startup, registrations build an `IServiceProvider` with compiled activators. Per request, the scope is created; the framework resolves the controller's constructor graph — each dependency resolved by lifetime rules — then disposes the scope (and all `IDisposable` scoped services) when the response completes. Missing registration fails at resolve time; validate scopes in development catch lifetime bugs.

Q. Where do AutoMapper-style mappers hurt, and what's the alternative?

A. Runtime reflection mapping hides broken mappings until runtime, resists trimming/AOT, and makes 'find usages' lie. Alternatives: Mapperly or manual mapping — source-generated, compile-time checked, zero reflection. Senior stance: mapping is boring code worth seeing; magic that hides a projection bug in production isn't worth the keystrokes saved.

Q. How do you serve static files securely and efficiently?

A. UseStaticFiles early to short-circuit the pipeline; set cache headers with fingerprinted filenames for immutable caching; in .NET 9+, MapStaticAssets adds compression and ETags automatically. Never enable directory browsing in production, don't serve from user-upload paths without validation, and put a CDN in front for anything at scale.

Q. A singleton service intermittently throws 'Cannot access a disposed object' around DbContext. Explain and fix.

A. Captive dependency: the singleton captured a scoped DbContext (directly or via another service) whose scope ended — first request wires it, later requests use the corpse. Fix: inject IServiceScopeFactory or IDbContextFactory, create a scope/context per operation. Enable scope validation in development builds — it catches this at startup instead of in production.

Q. Middleware you added runs on responses but its changes never reach the client sometimes. Why?

A. The response had already started — headers flushed — before your code on the return path ran; ASP.NET Core throws or ignores header mutations after first byte. Common with streaming or large responses. Fix: act before calling next, use `OnStarting` callbacks for headers, or buffer deliberately if you must inspect the body — accepting the memory cost that implies.

Q. Your endpoint accepts IFormFile uploads to 500MB and the pod OOMs. What's wrong with the design?

A. IFormFile with default binding buffers to memory/disk before your code runs, and model binding reads the whole form. For large uploads: stream instead — read MultipartReader sections directly to blob storage, raise only the specific limits needed, and enforce content-length early. Better architecture: pre-signed upload URLs straight to storage so the API never proxies the bytes.

Q. How do you implement per-tenant behavior cleanly — different limits, features, connections?

A. Resolve tenant once in early middleware (host/header/claim) into a scoped `ITenantContext` everything else injects. Tenant-specific options via `IOptionsSnapshot` with named/keyed configuration; keyed DI services (.NET 8+) for divergent implementations; per-tenant rate-limit partitions; connection selection in a `DbContext` factory. The anti-pattern: tenant if-statements scattered through handlers.

Q. Kestrel behind Nginx/ALB: what actually needs configuring that people miss?

A. ForwardedHeaders middleware — otherwise Scheme is http and RemoteIp is the proxy, breaking redirects, cookies, and rate limits keyed by IP; configure KnownProxies/Networks so headers can't be spoofed. Also: body size limits at both layers must agree, timeouts (proxy shorter than Kestrel causes mystery 502s), and HTTP/2 end-to-end if you use gRPC.

Q. A filter, a middleware, and an `ExceptionHandler` could each solve your cross-cutting need. How do you choose?

A. Middleware for protocol-level concerns applying to everything — auth, correlation IDs, compression — no MVC context. Filters when you need endpoint/action context: model state, action arguments, per-controller opt-in. `ExceptionHandler` specifically for typed error-to-`ProblemDetails` mapping. Wrong tool symptoms: middleware parsing route data manually, or filters doing what one middleware line could.

Q. Startup takes 40 seconds in production. Where do you look?

A. Measure first — event counters or simple stopwatch logging around Program.cs phases. Usual offenders: synchronous I/O at startup (config from remote stores, migrations, cache warming), assembly-scanning DI registration over hundreds of assemblies, EF model building for huge models (use compiled models), and missing ReadyToRun. Move heavy warmup to background with readiness gating traffic until done.

Q. How do you make config changes take effect without redeploying, safely?

A. Config provider with reload (Azure App Configuration, Consul, or file-watching in K8s ConfigMaps) + IOptionMonitor for services that must react. Safety: validate on reload (ValidateDataAnnotations plus custom rules) so bad config rejects rather than applies; log config version on change; and treat feature flags as the primary runtime lever — they have targeting, audit, and kill-switch semantics config lacks.

Q. Two endpoints need the same 'current user' data; one team hits the DB in a filter, another in the handler. Arbitrate.

A. Neither duplicated: load once per request into a scoped `CurrentUserContext` — populated lazily on first access so endpoints that don't need it pay nothing, cached for the request's duration. Filter vs handler placement is the wrong debate; the right question is 'what owns the lifecycle of request-scoped state' and the answer is a scoped service, tested independently.

Q. What does [ApiController] actually do — and when has its magic hurt you?

A. Enables attribute routing requirement, automatic 400 on invalid ModelState, binding source inference ([FromBody] for complex types), and ProblemDetails for error status codes. It hurts when: you wanted custom validation response shape (configure InvalidModelStateResponseFactory), inference binds from the wrong source, or auto-400 hides that validation ran before your logging. Know the switches; don't cargo-cult the attribute.

That's a wrap.

20 ASP.NET Core questions that actually come up in senior .NET interviews — with the answers experienced engineers give.

NEXT IN THE SERIES

API Design

Found this useful?

Follow for the rest of the series,
and save this post for your interview prep.